

第一次测试 6 题解题报告

李博文

北洋机甲算法组 2027 赛季培训

2026 年 10 月 7 日

摘要

本报告记录我对北洋机甲算法组第一次测试 6 道题的实现思路与验证结果：前三题用 C++ 复现 C 字符串库函数 (`my_strlen`、`my_strcat`、`my_strstr`)，后三题是图像处理中的常用函数 (`rgb2gray`、`resize`、`hist_eq`)。所有代码在 WSL Ubuntu 22.04 下用 CMake + g++ 11.4 编译；图像结果由程序生成，并与仓库中的参考答案逐像素比对。

目录

1	<code>my_strlen</code> : 自己实现字符串长度	2
2	<code>my_strcat</code> : 字符串拼接	2
3	<code>my_strstr</code> : 子串查找	3
4	<code>rgb2gray</code> : 彩色图转灰度图	4
5	<code>resize</code> : 双线性插值缩放	5
6	<code>hist_eq</code> : 直方图均衡化	6
	附录: 如何验证	7
7	附加题一: 多目标下的最佳打击目标选择	7
8	附加题二: 迷宫最短路径 (继承 + 广度优先搜索)	9

1 my_strlen: 自己实现字符串长度

1.1 题目

统计一个以 '\0' 结尾的字符串有多少个字符，不调用现成的 strlen。

1.2 思路

从首地址开始，只要当前字符不是结束符就往后走一步并计数；遇到结束符时计数值就是长度。

1.3 代码

```
1 int my_strlen(char* str) {  
2     int len = 0;  
3     while (str[len] != '\0') ++len;  
4     return len;  
5 }
```

1.4 结果

test_strlen() 输出: **通过** (对 "123456"、""、"hello world!" 三个样例都与标准库结果一致)。

2 my_strcat: 字符串拼接

2.1 题目

把传入的两个字符串首尾拼接在一起，组成一个新的字符串，并确保以 '\0' 结尾

2.2 思路

让传进来的 *str1 走到 str1 的结束符上，再利用循环把 str2 的字符值逐个赋给 *str1，二者每轮循环自增，最后在拼接字符串末尾加上 '\0'，过程中要保证 str1 申请的内存空间足够大

2.3 代码

```
1 void my_strcat (char* str_1, char* str_2){  
2     while (*str_1 != '\0') str_1++;  
3     while (*str_2 != '\0') {  
4         *str_1 = *str_2;  
5         str_1++;  
6         str_2++;  
7     }  
8     *str_1 = '\0';
```

```
9 }
```

2.4 结果

test_strcat() 输出: **通过** (先用标准库 strcat 算出期望结果, 再用 strcmp 比较整串, 说明结束符 '\0' 也写对了)。

3 my_strstr: 子串查找

3.1 题目

在一个字符串中查找是否有与目标子串完全相同的部分, 若有, 返回这部分的首地址; 若无, 返回 nullptr

3.2 思路

通过循环找出第一个相同的字符, 并将之后的每个字符进行对比, 若完全相同则返回第一个相同字符在字符串中的地址; 若出现不同则 break, 寻找下一个“第一个相同的字符”, 一直没找到返回 nullptr。过程中需保证字符串排查余下的长度大于等于子串长度特殊情况: 目标子串为空时, 直接返回字符串首地址

3.3 代码

```
1 char* my_strstr(char* s, char* p) {  
2     int len_s = my_strlen(s), len_p = my_strlen(p);  
3     if (len_p == 0) return s;  
4     for (int i = 0; i < len_s - len_p + 1; ++i) {  
5         for (int j = 0; j < len_p; ++j) {  
6             if (s[i + j] != p[j]) break;  
7             if (j == len_p - 1) return &s[i];  
8         }  
9     }  
10    return 0;  
11 }
```

3.4 结果

test_strstr() 输出: **通过** (对 "wjsfjdvioz"、"qqqqq"、"j" 三个子串, 返回的指针都与标准库 strstr 相同)。

4 rgb2gray: 彩色图转灰度图

4.1 题目

将传入的彩色图转为灰度图

4.2 思路

彩色图在内存里按 R,G,B 三通道连续存放,第 (x, y) 个像素的三个通道位于 $\text{in}[(y*w+x)*3]$ 、 $+1$ 、 $+2$ 。灰度公式为

$$V = 0.2989 R + 0.5870 G + 0.1140 B \quad (1)$$

以纵坐标自增为外层循环,横坐标自增为内层循环,遍历图片的每个像素,并用灰度公式进行转化

4.3 代码

```
1 void rgb2gray(float* in, float* out, int h, int w) {  
2     for (int y = 0; y < h; ++y){  
3         for (int x = 0; x < w; ++x){  
4             int index = y*w + x;  
5             out[index] = in[index*3]*0.2989 + in[index*3 + 1]*0.5870 + in[index*3 +  
6                 2]*0.1140;  
7         }  
8     }
```

4.4 结果

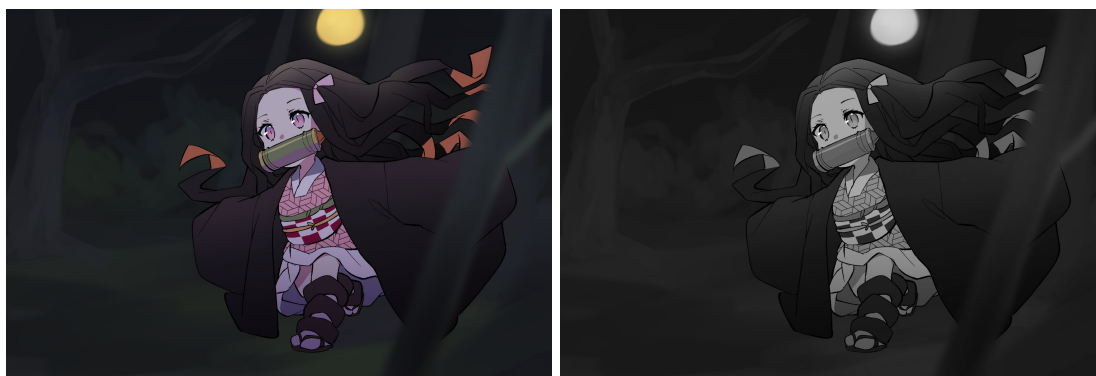


图 1: 左: 输入彩色图; 右: 我的程序输出的灰度图

5 resize: 双线性插值缩放

5.1 题目

通过双线性插值法，实现图片的缩放函数

5.2 思路

$$x2 - xx - x1v = v1 * \frac{x2 - x1}{x2 - x1} + v2 * \frac{x2 - x1}{x2 - x1} \quad (2)$$

1. 对于每个 dst 中的像素点 (x, y)，先计算出其在 src 中的坐标 float(x0,y0)，2. 然后计算出其在 src 中的四个邻居点: x1 = static_cast<int>(x0), y1 = static_cast<int>(y0) 上面这样可以直接将 float 通过下取整的方式转换为 int，剩下三个邻居就好找了 3. 用双线性插值法，用四个邻居的 v 值算出插值，再赋给 dst 中的像素点 4. 注意上面的方法中，四个邻居点的坐标可能会超出 src 的范围，所以需要对其进行边界检查

5.3 代码

```

1 void resize(float* in, float* out, int h, int w, int c, float scale) {
2     int new_h = h * scale, new_w = w * scale;
3     for (int y = 0; y < new_h; ++y){
4         for (int x = 0; x < new_w; ++x){
5             for (int ch = 0; ch < c; ++ch){
6                 float x0 = static_cast<float>(x) / scale, y0 = static_cast<float>(y) / scale;
7                 int x1 = static_cast<int>(x0), int y1 = static_cast<int>(y0);
8                 if(x1 >= w - 1) x1 = w - 2;
9                 if(y1 >= h - 1) y1 = h - 2;
10                int x2 = x1 + 1, int y2 = y1 + 1;
11                float q1 = in[(y1*w + x1)*c + ch] * (x2 - x0) / (x2 - x1)
12                    + in[(y1*w + x2)*c + ch] * (x0 - x1) / (x2 - x1);
13                float q2 = in[(y2*w + x1)*c + ch] * (x2 - x0) / (x2 - x1)
14                    + in[(y2*w + x2)*c + ch] * (x0 - x1) / (x2 - x1);
15                float Q = q1 * (y2 - y0) / (y2 - y1) + q2 * (y0 - y1) / (y2 - y1);
16                out[(y*new_w + x)*c + ch] = Q;
17            }
18        }
19    }
20 }

```

5.4 结果

test_resize() 生成 output0.jpg (1620×1098, 2/3 倍) 与 output1.jpg (4860×3296, 2 倍): 尺寸与 h*scale、w*scale 一致，色彩统计与输入一致，边界无越界噪点。



图 2: 左: 输入原图; 中: 输入缩小图; 右: 输入放大图

6 hist_eq: 直方图均衡化

6.1 题目

让传入的灰度图片灰度级分布更均匀，即增强图片对比度

6.2 思路

1. 建好数桶，记录每个灰度数出现的频次
2. 累计每个灰度级及其以下的灰度级的总频次
3. 依据每个灰度级及其以下的灰度级的总频次在总像素数的占比，映射出新的均衡的灰度级分布
4. 把新的灰度级赋给对应像素

6.3 代码

```
1 void hist_eq(float* in, int h, int w) {  
2     int hist[256] = {0};  
3     for (int i = 0; i < h*w; ++i) ++hist[static_cast<int>(in[i])];  
4     int cdf[256] = {0};  
5     cdf[0] = hist[0];  
6     for (int i = 1; i < 256; ++i) cdf[i] = cdf[i - 1] + hist[i];  
7     float map[256] = {0};  
8     for (int i = 0; i < 256; ++i) map[i] = static_cast<float>(cdf[i]) / static_cast<  
9         float>(h * w) * 255.0f;;  
10    for (int i = 0; i < h*w; ++i) in[i] = map[static_cast<int>(in[i])];  
11 }
```

6.4 结果



图 3: 左: 输入目标图; 右: 输入修改图

`test_hist_eq()` 生成 `output.jpg`, 与参考图 `answer.jpg` 逐像素平均差 $0.72/255$; 均衡化后全图均值 133 (接近中灰 128), 说明直方图确已被拉平。

附录：如何验证

```
1 cd ~/tjurm2027-test1
2 cmake --build build -j$(nproc)
3 cd build && ./tjurm-test # 注意必须在 build/ 里运行 (图片路径是 ../ 相对的)
```

图像题的结果可用 `git status` 做字节级验收：产出图若与仓库里的参考版本不同，它们会显示为修改状态。

7 附加题一：多目标下的最佳打击目标选择

7.1 题目

模拟 RoboMaster 自瞄中的目标选择：记录 4 个敌方目标的二维坐标与兵种 ID，**选出距离准心（屏幕中心，即原点）最近的目标并输出其兵种 ID**。要求采用面向对象方式设计两个类：`Enemy`（坐标 + 兵种 ID + 设置/获取函数）与 `Target`（内含 4 个 `Enemy` 对象的数组，并能选出最佳目标）。

7.2 思路

1. `Enemy` 管“一个目标”：id、x、y 都是私有成员，只能通过公开的 `Set / getX / getY / getid` 读写。数据私有、接口公开，外部就无法把对象改成非法状态。
2. `Target` 管“一组目标”：内部持有 `Enemy enemies[4]` 对象数组；`setEnemy(i, ...)` 把数据转交给第 i 个 `Enemy`，`choose()` 遍历数组选出离准心最近的那个。

3. **比较距离时只比较距离的平方**: $d^2 = x^2 + y^2$ 与 $d = \sqrt{x^2 + y^2}$ 单调性完全一致, 因此省掉一次开方, 也避免浮点误差。
4. **默认构造函数**: Enemy 必须有无参构造函数, 否则 `Enemy enemies[4];` 无法编译——数组里每个元素都要能“凭空造出来”。

7.3 代码

```

1  class Enemy {
2  public:
3      Enemy() : id('?'), x_(0), y_(0) {}           // 默认构造函数: 对象数组需要它
4      void Set(char ID, double x, double y) { id = ID; x_ = x; y_ = y; }
5      double getX() const { return x_; }           // const: 承诺不修改自身
6      double getY() const { return y_; }
7      char getId() const { return id; }
8  private:
9      char id; double x_, y_;                       // 数据私有, 外部只能走公开接口
10 };
11
12 class Target {
13 public:
14     void setEnemy(int index, char ID, double x, double y) {
15         enemies[index].Set(ID, x, y);             // 转交给对应的 Enemy 对象
16     }
17     char choose() const {
18         int best = 0;
19         for (int i = 1; i < 4; ++i) {
20             double d = enemies[i].getX() * enemies[i].getX()
21                 + enemies[i].getY() * enemies[i].getY();
22             double db = enemies[best].getX() * enemies[best].getX()
23                 + enemies[best].getY() * enemies[best].getY();
24             if (d < db) best = i;                   // 只比平方, 省一次开方
25         }
26         return enemies[best].getId();
27     }
28 private:
29     Enemy enemies[4];
30 };
31
32 int main() {
33     Target target;
34     for (int i = 0; i < 4; i++) {
35         double x, y; char id;
36         cout << "请输入第 " << i + 1 << " 个目标的兵种ID和坐标(x y): ";
37         cin >> id >> x >> y;

```

```

38     target.setEnemy(i, id, x, y);           // 通过 Target 的接口写入
39 }
40 cout << target.choose() << endl;         // 输出最佳目标的兵种 ID
41 return 0;
42 }
```

7.4 结果

用题目附带的 `visual/test.txt` 中 5 组用例测试（第 5 组含小数坐标），实际输出与期望全部一致：

表 1: 附加题一：5 组用例的验证结果

用例	四个目标（ID 与坐标）	期望 / 实际
1	H(-9,3) E(1,4) S(-3,1) I(7,-3)	S / S
2	H(0,5) S(-4,0) T(0,-2) I(6,0)	T / T
3	H(7,2) S(-5,5) T(2,-1) I(8,8)	T / T
4	H(10,10) S(-8,6) T(5,-9) I(1,1)	I / I
5	H(2.5,3.5) S(-1.5,0.5) T(4,-1) I(0.5,-0.5)	I / I

8 附加题二：迷宫最短路径（继承 + 广度优先搜索）

8.1 题目

给定一张 14×71 的地图，# 表示障碍、. 表示空地，求机器人从左上到右下的最短路径，把路径用 C 标出并打印整张地图；机器人只能四向移动。**加分项**：考虑机器人有体积、需要 3×3 的空地才能安全移动，要求给出满足该约束的路径。

8.2 思路

1. **继承**：写一个 `MAP_BASE` 的子类 `SHORTEST_PATH`，直接使用基类 `protected` 的地图 `map_in` 与基类的 `print()`，自己额外加一个 `dist` 数组记录步数。
2. **为什么用 BFS**：队列先进先出，程序按“离起点的步数”一层层向外扩展——先看所有 1 步能到的格子、再看 2 步能到的……所以**第一次到达某格时，走的一定是最少步数**。若改用栈（DFS），会一头扎到底，第一次到达不保证最短。
3. **入队时就标记 dist**（而不是出队时）：否则同一个格子会被它的多个邻居反复入队，队列膨胀甚至死循环。
4. **回溯**：BFS 只知道步数、不知道路线。从终点反着走，每步都找“`dist` 恰好比当前小 1”的邻居，一路回到起点，沿途标 C——这样不需要额外的 `from` 数组。

5. **加分项**：把“这一格是空地”换成“以这一格为中心的 3×3 全是空地”。注意地图两个角 (0,0)、(13,70) 本身就是墙，真正的起终点是 (1,1) 与 (12,69)；而 (1,1) 紧贴边界、它的 3×3 邻域里必然有墙，**机器人根本放不下**——所以加分版的安全起终点是 (2,2) 与 (11,68)。

8.3 代码

```

1  class SHORTEST_PATH : public MAP_BASE {
2  public:
3      bool is_3x3_clear(int r, int c) {           // 机器人中心在 (r,c) 时能否安全放置
4          for (int i = -1; i <= 1; ++i)
5              for (int j = -1; j <= 1; ++j)
6                  if (map_in[r + i][c + j] == '#') return false;
7          return true;
8      }
9      void path() {
10         const int W = 70, H = 14;
11         int dist[H][W];
12         for (int r = 0; r < H; ++r)
13             for (int c = 0; c < W; ++c) dist[r][c] = -1;
14
15         queue<pair<int,int>> bfs;                // 队列先进先出 → 层序扩展 → 最短路
16         dist[2][2] = 0;                        // 加分版的安全起点
17         bfs.push({2, 2});
18         int dr[4] = {-1, 1, 0, 0};            // 四向：上、下、左、右
19         int dc[4] = {0, 0, -1, 1};
20
21         while (!bfs.empty()) {
22             pair<int,int> cur = bfs.front(); bfs.pop();
23             if (cur.first == 11 && cur.second == 68) break;
24             for (int i = 0; i < 4; ++i) {
25                 int nr = cur.first + dr[i], nc = cur.second + dc[i];
26                 if (nr >= 2 && nr <= 11 && nc >= 2 && nc <= 68 &&
27                     is_3x3_clear(nr, nc) && dist[nr][nc] == -1) {
28                     bfs.push({nr, nc});
29                     dist[nr][nc] = dist[cur.first][cur.second] + 1; // 入队即标记
30                 }
31             }
32         }
33
34         int r = 11, c = 68;                    // 回溯：沿 dist 下坡走回起点
35         while (r != 2 || c != 2) {
36             bool found = false;
37             for (int i = 0; i < 4; ++i) {
38                 map_in[r][c] = 'C';

```

```

39         int nr = r + dr[i], nc = c + dc[i];
40         if (nr >= 2 && nr <= 11 && nc >= 2 && nc <= 68 &&
41             is_3x3_clear(nr, nc) && dist[nr][nc] == dist[r][c] - 1) {
42             r = nr; c = nc; found = true;
43         }
44     }
45     if (!found) break;           // 保险：避免死循环
46 }
47 map_in[2][2] = 'C';
48 }
49 };
50
51 int main() {
52     SHORTEST_PATH navigator;
53     navigator.path();
54     navigator.print();           // 基类提供的打印函数
55 }

```

8.4 结果

程序输出如下（加分版：机器人全程停在 3×3 的空地上）：

```

1 #####
2 #.....#####.#.....#
3 #.CC.....#####.CCCCCCCCCCCC.....#####
4 #.CC.....#####.CC.....CC.###.###.....#####
5 ##.CC.....#.#####.CC.....CC.....#
6 ###.CC.....CCCCCCC.....CCCCCCCCCCCC.....#
7 #....CC.###....CC.....#####.CC.###....#
8 #....CC.##....CC.##.....#####.CC.....#
9 #....CC.....CC.....CCCC.....#
10 #....CCCCCCCC.....##.....CC.#####
11 #.....CC.....#
12 #####.....#####.#####.CCCCCCC.#
13 #.....#####.#####
14 #####

```

我把输出交给脚本做了 4 项不变量检查，全部通过：

1. 除路径外，地图其余格子与题目给定地图**完全一致**（没有误改）；
2. C 的个数为 **90**，对应 **89** 次移动，与独立实现的 BFS 结果一致；
3. C 构成一条**连续的四连通路**径（每个 C 的 C 邻居数不超过 2，无断裂、无分叉），且从起点能遍历全部 C；
4. 路径上**每一格的 3×3 邻域全为空地**，满足加分项要求。

作为对照：若去掉 3×3 约束，同一张地图上的最短路径为 **80** 个格子（79 次移动），起终点是 (1,1) 与 (12,69)。两个数字由同一套 BFS 得到，区别只在于“能否走”的判定条件。

表 2: 附加题二：两种约束下的最短路径

版本	起点 $\rightarrow$ 终点	路径格子数	移动步数
基础（仅要求是空地）	$(1, 1) \rightarrow (12, 69)$	80	79
加分（每格 3×3 全空）	$(2, 2) \rightarrow (11, 68)$	90	89